

Processing excel files $\rightarrow$ Apache POI

Interview question: How will you process large excel files in java?

$\Rightarrow$ So to process excel files in java, we use "Apache POI" library which provides APIs to read, create, manipulate ~~Manipu~~ Microsoft excel files.

Apache POI provides `XSSFWorkbook` that loads the entire file into memory. It allows us to access all data in a spreadsheet but can cause memory issues for large files.

So for large excel file we use Apache POI streaming based SAX API, which works in event driven manner, reads file row by row (one row at a time) without loading entire row into memory, which reduces memory usage.

duplicates & count in huge excel



① filepath

↳ abstract class of apache poi

② OPCPackage opcPackage = OPCPackage.open(file);

↳

a) opens excel file as open xml package

b) .xlsx are zip files containing xml

c) OPCPackage allows POI to access internal xml parts safely.

* d) needed because SAX works on XML stream - this step prepares file for streaming.

* This opens the excel file as an OpenXML package so that its internal xml structure can be accessed. - required for SAX streaming.

③ XSSFReader reader = new XSSFReader(opcPackage);

a) creates reader for excel

b) allows streaming access to excel

c) without loading entire excel file into memory.

④ InputStream sheet = reader.getSheetsData().next()

↳
↓
iterator of sheets

↳ 1st sheet

⑤ `Map<String, Long> countMap = new HashMap<>();`

⑥ `XMLReader parser = XMLReaderFactory.createXMLReader();`

- ↳ a) creates ^{sax} xml parser
- b) event driven
- c) reads xml sequentially

IMP

⑦ `parser.setContentHandler(new SheetHandler(countMap));`

- ↳ a) tells sax where to send data
- b) Registers SheetHandler as listener
- c) calls `startElement()` → business logic
`characters()`
`EndElement()`

.... SAX does not return data
 it fires event

processes Excel file ... by a handler provided
 ... processes excel file.

⑧ `parser.parse(new InputStream(sheet));`

- ↳ a) starts reading excel
 - b) events are fired -- handler method
- start,
char,
end

∴ Starts parsing the excel file and triggers handlers.

SheetHandler

called when cell is started -- where we identify target col

StartElement

targetCol false

CellValue StringBuilder

(1) select cells, rows/cols etc

↳ targetCol

called when data in cell is read and appended

characters

get text content inside cell & append

cell ends and we

update the count map

↳ End

targetCol & end tag

↳ cell value

* `countMap.put(value, countMap.getOrDefault(value, 0) + 1)`

allows to count duplicates immediately while reading file row by row

String filePath

OP(Package (filePath))

OP(Package opPackage = OP(Package.open(file))

XEEF Reader (opPackage)

InputStream sheet = reader.getSheetsData.next()

XML Reader parser

· setContentHandler (SheetHandler)

↳ startE
char

EndE

parser.parse(sheet)

entrySet()

(out map.stream())

· filter (e -> e.getValue > 1)

· forEach (e -> sort (e.key -> e.value))

sheet close

opc close

To process normal size excel



psvm throws Exception

```
FileInputStream fileInputStream = new FileInputStream("file");
```

```
XSSFWorkbook workbook = new XSSFWorkbook(fileInputStream);
```

↳ loads entire excel file

```
Sheet sheet = workbook.getSheetAt(0);
```

↳ 1st spreadsheet.

```
DataFormatter dataFormatter = new DataFormatter();
```

```
Map<String, Long> countMap = new HashMap<>();
```

```
for (Row row : sheet) {
```

```
    Cell cell = row.getCell(0);
```

```
    if (cell == null) continue;
```

```
    String cellContent = dataFormatter.formatCellValue(cell);
```

```
    if (cellContent.isEmpty()) continue;
```

```
    countMap.put(cellContent, countMap.getOrDefault(  
        cellContent, 0L) + 1);
```

```
}
```

countMap.entrySet()

- stream()
- filter (e → e.getValue() > 1)
- forEach (e → sout (e.getKey() + " → " + e.getValue()))

workbook.close();

fileInputStream.close();

list

put (cellCount, countMap.getOrDefault (cellCount, 0) + 1);

3
5
3

3 : 0 + 1 = 1
5 : 0 + 1 = 1
3 : 1 + 1 = 2

countMap = [~~3:1~~, 5:1
3:2]

API design in SpringBoot Microservice



We will have a common package

common

└ response

BaseResponse.java

ResponseBuilder.java

└ exception

GlobalExceptionHandler.java

ResourceNotFoundException.java

@Getter

@Setter

@Builder

@NoArgsConstructor

@AllArgsConstructor

```
public class BaseResponse<T> {
```

```
    private int status;
```

```
    private String message;
```

```
    private T data;
```

```
    private List<Map<String, Object>> errors;
```

```
}
```

LL

consistent response for all endpoints in our microservice

HTTP code

Headers

```
{ status  
  msg  
  date:  
  error;
```



```
public class ResponseBuilder {
```

```
① public static <T> ResponseEntity<BaseResponse<T>> success (
    HttpStatus status, String message, T data) {
```

```
    return ResponseEntity.status(status)
```

```
        .body(BaseResponse.<T>builder(
```

```
            .status(status.value())
```

```
            .message(message)
```

```
            .data(data)
```

```
            .build()));
```

```
}
```

```
② public static <T> ResponseEntity<BaseResponse<T>> error (
    HttpStatus status, String message, List<Map<String, Object>> errors) {
```

```
    BaseResponse<T> baseResponse = new BaseResponse(
```

```
        status.value(),
```

```
        message,
```

```
        null,
```

```
        errors);
```

```
    return ResponseEntity.status(status).body(baseResponse);
```

```
}
```

Custom Exception

```
public class ResourceNotFoundException extends RuntimeException
```

```
{  
    public ResourceNotFoundException (String msg) {
```

```
        super (msg);
```

```
    }
```

```
}
```

Spring web, handle exception globally

* @RestControllerAdvice => in Spring boot REST applications across all controllers

```
public class GlobalExceptionHandler {
```

```
@ExceptionHandler (ResourceNotFoundException.class)
```

```
public ResponseEntity < BaseResponse < Object > > handleResourceNotFound  
( ResourceNotFoundException exception, HttpServletRequest request )
```

```
{  
    return Response Builder . error (
```

```
        HttpStatus . NOT_FOUND,
```

```
        exception.getMessage();
```

```
        List.of ( Map.of (
```

maintained

across

org & client

```
            "code", "RESOURCE-NOT-FOUND",
```

```
            "message", "no resource",
```

```
            "path", request.getRequestURI()
```

```
        )
```

```
    );
```

```
}
```

Spring cloud



SERVICE Registry & Eureka

lets say we have 2 microservices mca mcb
and then how can the communicate with each other
using host and port?

what if we have multiple instances of it

lie mca1 mcb1 mcl1
mca2 mcb2 mcl2

So for scalability & fault tolerance we use &
Service Registry discoverability

① when micro service starts it register itself to
service registry

② register its names + ip + port.

③ then it allows mcs to dynamically
discover each other w/o hardcoded ip/port

∴ Eureka is a central component which maintains a
list of microservices.. allows them to ^{discover each other} communicate
dynamically without hardcoded network (ip/port)

Client side discovery

- 1) ^{service} client registers itself
- 2) calling service:
 - fetches list
 - chooses instance (load balancer)
 - calls service directly

eg: Eureka

Server side discovery

- eg → K8s / ALB
 - client calls LB
 - LB → healthy service
 - ↳ request

So Eureka ... client side discovery
 client retrieves list of available instances
 from the registry and perform load balancing itself



LIFECYCLE

Register

- service starts

- sends: Service name

IP

port

- eureka stores it

Renew (Heartbeat)

~ 30 sec

Deregister

→ gracefully shutdown

→ service notifies eureka

- crash - eureka removes

service after timeout

important

1) Eureka = service registry only

it stores => A-Service (service name) → [A1, A2, A3] (list of instances)

2) internal communication between microservices

@LoadBalanced => Spring cloud load balancer

public RestTemplate restTemplate() {

return new RestTemplate();

}

older ribbon

A-service → restTemplate call with service name

↓

Called

@LoadBalancer intercepts

↓

ask eureka for instances

Loadbalancer

↓ pick one instance

access service instance

B1

Client Side

Load Balancing

- * `@LoadBalanced` tells spring to intercept rest calls
 - get list of instances from eureka
 - uses spring loadbalancer to pick one instance.



* `RestTemplate` $\Rightarrow$ synchronous http client to consume RESTful web services.

Making a Spring Eureka Service Registry

pom $\Rightarrow$

`spring-cloud-starter-netflix-eureka-server`

code $\Rightarrow$

`@SpringBootApplication`

`@EnableEurekaServer` $\leftarrow$

`public class EurekaServerApplication()`

`psvm() {`

`SpringApp.run`

`}`

`}`

for single node.

application props:

`name`

`port`

`eureka.client.fetch-registry = false`

`eureka.client.register-with-eureka = false`

`eureka.client.service-url.defaultzone = ip:port / eureka`

fetches list of instances

registers itself on eureka

Peer aware eureka 3-node cluster.

- Multiple Eureka Servers registers with each other & replicate registry data
- provides high availability.

now 3 eureka $\Rightarrow$ diff ports 8761, 8762, 8763

- each is a server
- client
- registers itself to other two servers.

* eureka client registers with eureka = true
fetch-registry = true.

* app-prop in microservice

port

name

discoveryclient.defaultzone : E1 ip1, E2 ip2, E3 ip3

register with eureka = true.

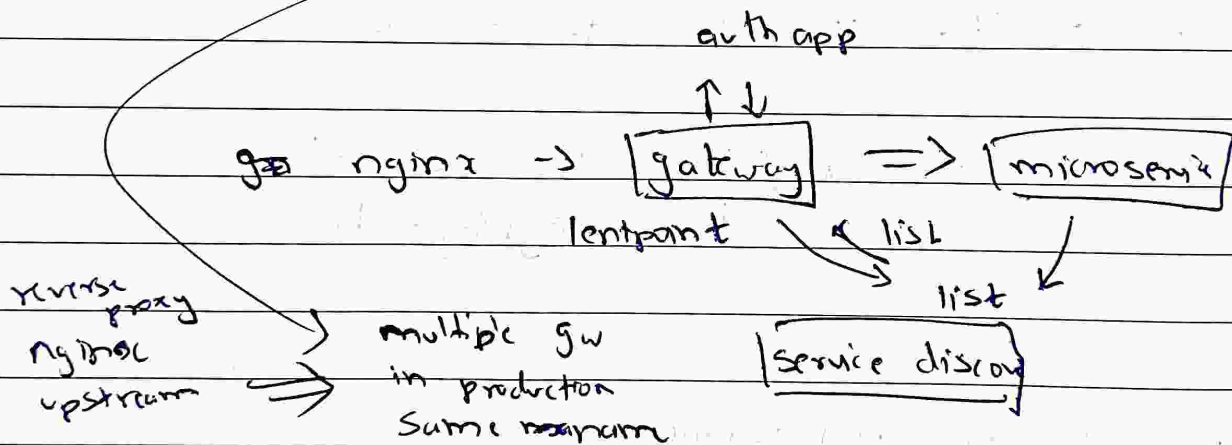
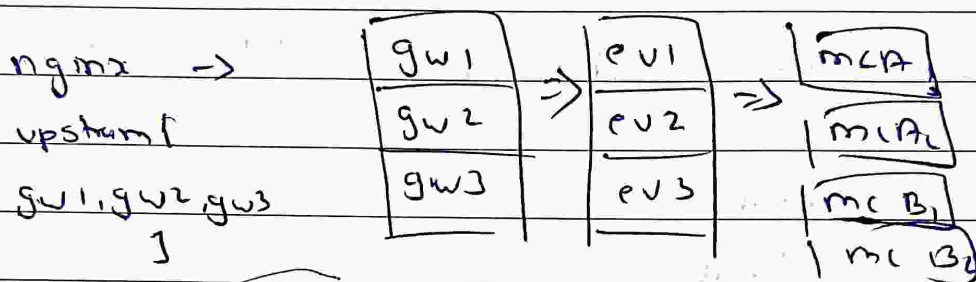
fetch registry = true

Spring API gateway



provide

- Single point of entry for all client requests
- ~~ph~~ handles ^{auth, auth} security, routing, rate limiting
- hides internal service topology
- routes to correct microservice
- integrates with service registry eureka
- load balances across instances



gateway.props

name

port

eureka-client.service-url.defaultZone=eurekaIP:port

* `spring.cloud.gateway.routes[0].id` = transaction-app (transaction)

`uri` = `lb://transactions`

`predicate` = `Path=/transactions/`

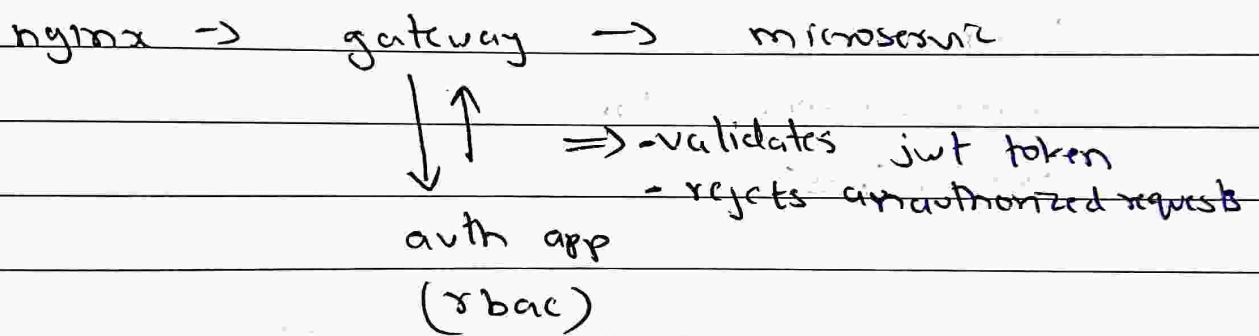
route this request only if url path starts with "/transactions"



* Filters :

* Cross cutting API concerns

allows centralized handling cross cutting features
concerns like authentication, logging, rate limiting,
and request transformation by modifying
request & response at gateway level.



rate limiting ?

JWT ?

auth ?

nginx

load balances in
nginx - distributes requests
to multiple api gateways
... availability



upstream api-gateway-cluster 1

least conn;

algo... route to least number of active connections

Server gwIP1:port maxfails:3 failtimeout 30

Server gwIP2:port -1-

Server gwIP3:port -1-

}

- default round robin

- least conn

- ip hash

Server {

listen 80;

server_name domain;

ssl things?

reverse proxy

location /transactions {

proxy_pass http://api-gateway-cluster;

}

nginx thing: High performance web server & reverse proxy
- used as reverse proxy, load balancer to
improve performance, scalability & availability.

SSL + nginx

what is SSL + TLS

- is a security protocol
- encrypts data between client & server

* Secures client-server communication using encryption & certificates, preventing man in the middle attacks

- ① client requests `https://domain`
- ② server sends ssl certificate
- ③ client verifies certificate via CA
- ④ handshake using public key,
Symmetric key for data transfer.
- ⑤ encryption communication starts

* SSL termination ... off-loading

- 1) client to nginx ... encrypted (https)
- 2) the traffic is then forwarded as unencrypted http to backend application server.

server: internal

* domain

- ssl certificate bind public key to a domain name, establishing trust.

- TLS is secure, more modern version of SSL.
depreciated



- wildcard tls certificate secures domain & all its subdomains

Config modularization - split configuration.

|etc/nginx

└ nginx.conf

└ conf.d

└ payment.conf

└ transactions.conf

└ upstream

└ api-gateway.conf

each microservice own nginx file

http {

include /etc/nginx/conf.d/*.conf;

}

systemctl start nginx.service
status
stop

nginx -t

nginx -s reload

Rewrite rules



* ↓ used to modify the request uri
before forwarding to backend.

```
location /orders {  
    rewrite ^/orders/?(.*)$ 1$1 last;  
    proxy-pass http://backend;  
}
```

client sends : /orders/create
backend receives : /create

location ~ ^/order/(abc|xyz)\$ {

only matches
abc, xyz
* is ignored.

proxy-pass http://backend;

}

Java Strings

1) String Literal

mostly used

```
String s1 = "Hello";
```

```
String s2 = "Hello";
```

```
s1 == s2 => true
```

- ① stored in string constant pool in heap
- ② if same value (object) already exists in SCP JVM reuses ... memory efficient.
- ③ == returns true if same value. ∴ both points to same object in memory.

2) String Object

```
String s2 = new String("Hello");
```

```
String s3 = new String("Hello");
```

```
s2 == s3 => false
```

- ① stored in heap outside SCP ∴ new objects is created
- ② if even if value already exist $\Rightarrow$ new()
- ③ less memory efficient.
- ④ == returns false even if same content because both references to different objects stored in memory.
equals() returns true.

Immutable

① Once a String literal or String Object is created, its value cannot be changed.

② Any modifications (concat, +, replace) creates a new string object instead of changing the ^{existing} String.

```
String s1 = "Hello";
```

```
s1.concat(" World");
```

cout(s1) => "Hello" because it points to s1 "Hello"

concat() creates a new object

String is immutable.

a) thread safe

b) reducing mem usage if same

c) immutability is a foundation of functional programming

* StringBuffer & StringBuilder are mutable.

① They modify the same object in memory instead of creating new one.

② Used internal character array ... grow & change

```
StringBuilder sb = new StringBuilder("Hello");
```

```
sb.concat(" World");
```

```
cout(sb) => "Hello World"
```

↳ modified same object.

String Builder

(1) non-thread safe

(2) faster...

due to no synchronization overhead.

(3) used in single-threaded env and performance

String Buffer

thread safe

slower

used in multi-threaded env data shared across thread.

Interview questions

1) What is String Pool?

- String Pool (String Constant Pool) is a special memory in heap to store string literals. So when new string literal is created, JVM checks the pool first if the value already exists it reuses the existing object instead of recreating it a new one. which improves memory efficiency & performance.

2) What is difference between String, StringBuilder & String Buffer?

(1) String is immutable. once created cannot be changed. if modified new object is created (mem heavy)

(2) StringBuilder & StringBuffer creates mutable ^{slow} string objects. ∴ if modified it changes same existing object. uses internal character array which can grow & change.

(b) StringBuffer is not thread safe, faster does not have synchronization.

String Buffer is thread safe, has synchronization, slightly slower because of it.

3) Why is String Immutable?

- Strings are immutable for security, thread safety and performance reasons.
- Immutability prevents modification of sensitive data like passwords.
- Allowing safe sharing across threads.
- And because of immutability ... ^{since} JVM used String constant pool to save string literals and reuses for strings with same values resulting in efficient memory usage & performance.

4) String literal vs String object?

- String literal is saved in String Constant pool in heap memory. When new string literal is created JVM first checks the pool if same value already exists it reuses it and ^{does} not create new object. New variable (reference) points to same existing object with same value.
 $\therefore == \text{true}$.

Created using keyword

- String Object `new String()` ^{always} creates a new object in heap memory. (even same value already exists)
`==` false.

5) When is StringBuffer better than String?

- when string need frequent modifications.
- when data needed to be shared across different threads (in multi threaded environment)
- Thread safety is required.

6) What does immutability mean?

- Immutability means once the object is created it cannot be change.
- If any modification is made, a new object is created.

7) How to create a immutable class?

- final class, with final variables/fields
 fields are initialized via constructor
 and no setter methods
 to prevent state change

```
final class Employee {
    private final name;
    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```


Array :- Collection of elements of same datatype
 Stored in contiguous memory locations.
 - allows random access using index $O(1)$
 - fixed size.

* Garbage Collection & memory management.

A) Memory management is how java

- (1) allocates memory when objects are created
- (2) how memory is used when program runs
- (3) free memory when object is no longer needed

(unreachable)*

B) Java memory areas (big picture)

When java program runs, memory is divided into 2 parts

(1) Stack memory :

- stores method calls () and local variables
- automatically freed when method finish execution
- fast, small thread specific.

(2) Heap memory :

- stores objects & class instances
- shared across threads
- managed by garbage collector.

example :

```
void foo() {
```

```
    int x = 10 ;
```

```
    Person p = new Person();
```

```
}
```

saved in stack

↳ reference 'p' on stack

Person object on heap

'p' points to object on heap

SO,

when method foo() ends;

- x is gone

- 'p' reference is gone

- * - Object Person is still on heap

but since variable reference 'p' which points to this object is gone... this object is

unreachable

∴ GC will clean it.

* What is Garbage Collection?

→ (1) Garbage collection is a process of automatically removing objects from heap memory which are no longer reachable.

(2) An object becomes a garbage when it has no active reference that points to it.

note:

- ① GC does not run immediately
- ② You cannot force GC.
- ③ You can only request it (`System.gc()`), jvm may ignore

Deep Dive

* Generational Heap Structure (Hotspot JVM)

core idea: more objects die young,
objects that survive longer tend to live longer

A] Young Generation:

where new objects are created, fast & frequent GC (minor GC)

① Eden Space:

- all new objects are allocated here
- when eden fills ... minor gc

② Survivor Space (S0, S1):

- objects that survive minor GC move here
- age increases with each GC cycle

B] Old Generation:

Stores long lived objects

- Objects moved here after surviving multiple minor GCs
- Collected by major / Full GC
- GC is slower & less frequent

* Interview questions

1) How garbage collection works in java?

(1) Garbage collection in java frees up heap memory by removing objects that are no longer reachable. JVM starts from GC roots like stack, stack variable & references, and keep all those objects (alive) that are reachable by them, anything else is eligible for collection.

(2) Java uses generational heap model. where objects are created in young generation and cleaned quickly fast, more frequent using minor GC. Objects that survive several GC are moved to old generation, which is collected slowly, less frequently.

(3) Garbage collections runs automatically and cannot be forced. This makes memory management safer but memory leaks still can happen if objects are unnecessarily kept referenced.

What algorithm JVM use for garbage collection?

(1) Mark and copy (in young generation):

- JVM marks live objects that are reachable via references, copies them to survivor space from eden space, and clears the rest at once.
- This is fast because most young objects die quickly and copying avoids fragmentation.

(2) Mark and Sweep (old generation)

in old gen

- JVM marks (live) to all reachable objects, then sweeps through memory removes unmarked (dead) objects.
- This frees memory but leaves gap between objects, causing fragmentation which can slow future allocation.

(3) Mark-Sweep-compact (old generation)

- After marking and sweeping JVM compacts the memory by moving the ^{live} objects together.
- Eliminates fragmentation, makes allocation faster.
- Slow & expensive.

Role of finalize() method

- ① The finalize() method is used in java primarily to perform clean up on non-memory resources like (file handling, network connections, database connections) just before object was removed from memory before garbage collector.
- ② But there was no guarantee when it will run or if it would even run. performance overhead, bugs ... deprecated since Java 9.
- ③ Modern java uses try-with-resources for safe and predictable resource management.

Autocloseable : Interface represents a resource that must be closed explicitly.

- must provide a close() method.
- to release resources like files, socket, db conn

try-with-resources :

- Feature in java automatically calls the close() method on resource that implements Autocloseable when try block finishes even exception occurs.

How can memory leak happen in java?

① Memory leak can happen in java when object is no longer need but is still referenced. So garbage collector cannot remove that object. Even though java has automatic garbage collection, objects which are reachable via gc roots will still remain in memory.

② Common causes are long-lived referenced like static fields/variables, growing collections where new objects are added but never removed, improper cache management. Such references will unintentionally keep the objects alive, causing the heap memory to grow over time and eventually causing performance issues leading to out of memory error.

```
public class CacheExample {
```

```
    private static List<String> cache = new ArrayList<>();
```

```
    public static void addData(String data) {  
        cache.add(data);
```

```
    }
```

```
} - add more faster than  
removal.
```

- obj stored in cache not properly treated
- Strong ref. like HashMap ~~list~~ arraylist
cache entries holding strong reference

What is JVM and why is it important?

- (1) Java virtual machine is a runtime engine that executes java byte code. ~~and~~ ~~provi~~
- (2) It is important because it enables platform independence - java code is compiled once ~~and~~ into byte code and can run on any machine with compatible jvm.
- (3) It also improves reliability and performance through features like garbage collection, memory management and just in time compilation, execution, hiding underlying OS and hw details from application.

Diff between JVM JRE JDK?

- JVM: core runtime executes byte code. mem management garbage collection, execution. It does not include libraries or development tools, only runs compiled code.
- JRE: JRE includes JVM along with libraries & supporting files needed to run java application. Installed on system where we need to only run java applications not develop them.
- JDK: includes JRE + development tools such as compiler, ^{other tools} debugger. - required for developing, debugging, compiling, running java apps.

Can a Java Application run without JRE?

- No. JRE includes

- JVM to execute java bytecode + mem mant + GC
- core libs to run java application

Is it possible to have JDK without JRE?

- No. JDK always include JRE.

development tools and compiler will also require runtime env to run java apps/program.

* JVM Architecture *

ClassLoader

• takes byte code (.class) ^{lazy} loads to mem

Runtime Data Areas

Class Area

metadata

class
method } names
field

Heap

reachable
Objects

+
gc

Stack

method() calls
local vars

each thread

method → new stack
frame
on top
+ pop
when done

PC register

current ins

executer
line

Native

method

stack

OS

JVM

* Execution Engine → JIT

executes bytecode

does not interpret
line by line

method called

repeatedly
→ native
machine code

(1927)

(1) Classloader Subsystem loads byte code (.class) and initializes it to memory.

(2) Then Runtime Data Area includes

Method/Class Area Metaspace

Heap

Stack

PC Register

Native Method Stack

(3) Execution engine executes bytecode using interpreter and just in time compiler for performance optimization

(3) Garbage collector automatically reclaims memory by removing unused objects.

What memory storages are available in JVM?

Which is faster, heap or stack, why?

(1) Stack is faster because allocation and deallocation of mem happens automatically with simple pointer movements.

There is no need of complex mgmt or gc.

(2) Heap is slower because object allocation is more complex and memory is managed by gc. involves reference lookups & gc pauses.

OOP

1) What is inheritance?

- Inheritance allows subclass to reuse ~~map~~ properties and behaviour of parent class using extends keyword enables code reusability and runtime polymorphism using method overriding.

↳ same method name
different behaviour

based on object of subclass

```
class Card {
```

```
    public void cardTxn() {
```

```
        "Card txn";
```

```
    }
```

```
class VisaCard extends Card {
```

```
    @Override
```

```
    public void cardTxn() {
```

```
        "visa txn";
```

```
    }
```

```
}
```

```
class MasterCard extends Card {
```

```
    @Override
```

```
    public void cardTxn() {
```

```
        "master";
```

```
    }
```

```
}
```


`Card c1 = new VisaCard();`
| |
 reference type Object type
 decides which methods which method implementation to
 are accessible execute.

Here, Card is Parent class and VisaCard is subclass that overrides CardTxn() method

So when ;

c1.CardTxn() is called java decides at runtime which overridden method to execute based on actual object..

∴ Runtime polymorphism

- ① note : "final" class, method, variable cannot be changed
- ② super keyword is used to call parents constructor, parents method, access parents variable.

Also... 2) Why is multiple inheritance is not allowed? ~~through class~~

- ① Java does not support multiple inheritance, because if two parent class have same method, java cant decide which one to call.. diamond problem.
- ② To avoid ambiguity caused by diamond ~~prog~~ problem
- ③ Instead, java supports multiple inheritance using interfaces, where conflicts must be explicitly resolved using by implementing a class.

Composition

- ① Composition is a design principle where a class contains another class as a field, enables code reuse through delegation.
- ② Delegation means a class used another class to perform certain behaviour by calling methods of that another class rather than inheriting it.
- ③ Composition is preferred over inheritance because it provides loose coupling, flexibility, better code maintainability.

Interview Question

3) Composition vs inheritance? + Q4

Ans Composition def

composition over inheritance

Inheritance def

Problems with inheritance

- 1) Tight coupling (to parent class)
- 2) Fragile Base class problem
- 3) Inherits unwanted methods
- 4) Behaviour fixed at compile time

Composition fixes

- 1) Behaviour changes at Runtime
- 2) Loose coupling
- 3) No unwanted methods
- 4) Safe from parent changes.

4) Composition over inheritance principle? many

Ans.

- (1) prefer building classed by combining objects rather than extending classes to reuse behaviour
- (2) heavily used in Spring
- (3) GOF design patterns
- (4) ques 3

5) Association, Aggregation, composition

(a) Association:

- (1) General relationship between 2 classes where one class uses or knows about another
- (2) Objects are related, but not responsible of each others lifecycle.

```
class Teacher {}
```

```
void teach() {}
```

```
}
```

```
class Student {
```

```
Teacher teacher;
```

```
}
```

Student can exists w/o teacher (null)

(b) Aggregation:

- (1) Objects have HAS-A relationship with weak ownership.
 - Parent uses child
 - Parent does not own lifecycle of child.
 - Child can exist independently.

```
class Employee {
    String name
}
```

```
class Department {
    List<Employee> employees;
}
```



- employee can exist w/o department
- vice versa.

* weak ownership

⑥ Composition

- Object has strong HAS-A relationship where the child object's lifecycle depends on the parent
- * constructor

Example of Composition + delegation
+ in Springboot

```
interface PaymentGateway {
    void pay();
}
```

⇒ Encap

```
class CardPaymentGateway implements PaymentGateway {
    public void pay() {}
}
```

```
class NetBankingPaymentGateway implements PaymentGateway {
    public void pay() {}
}
```

```
class Employee {  
    String name  
}
```

```
class Department {  
    List<Employee> employees;  
}
```



- employee can exist w/o department
- vice versa.

* weak ownership

E) Composition

- Object has strong HAS-A relationship where the child object's lifecycle depends on the parent
- * constructor

Example of Composition + delegation
+ in Springboot

```
interface PaymentGateway {  
    void in pay();  
}
```

⇒ ~~create~~

```
class CardPaymentGateway implements PaymentGateway {  
    public void pay() {}  
}
```

```
class NetBankingPaymentGateway implements PaymentGateway {  
    public void pay() {}  
}
```



```
class PaymentService {
```

```
    private PaymentGateway gateway;
```

```
    PaymentService (PaymentGateway gateway) {  
        this.gateway = gateway;  
    }
```

```
    void initiateTransaction () {  
        gateway.pay();  
    }
```

⇒ delegation

+ runtime polymorphism

```
}
```

```
PaymentService service = new PaymentService(  
    new CardPaymentGateway());
```

In Spring

```
class PaymentService {
```

```
    @Autowired
```

```
    private PaymentGateway gateway;
```

```
}
```

@RestController

```
class OrderController {
```

```
    @Autowired
```

```
    private OrderService orderService;
```

```
}
```

@Service

```
class OrderService {
```

```
    @Autowired
```

```
    private OrderRepository repository;
```

```
}
```

6) Polymorphism:

(1) Polymorphism in java allows same method to behave differently depending on object executing it or method parameters.

(2) It is achieved by method overloading at compile time and method overriding at runtime.

Card	VisaCard	MasterCard	
pay()	pay()	pay()	→ overriding

pay(cardnum)

pay(cardnum, payment processor)

→ over loading

overloading vs overriding

feature	overloading	overriding
time	compile-time	runtime
inheritance req.	no	yes
method signature	diff params	same signature
polymorphism type	static	dynamic

7) Encapsulation:

- ① Encapsulation is a design in java of wrapping data (fields/variables) and methods (behaviours) together in a class and restricting direct access to data using access modifiers.
- ② Implemented ~~used~~ using private fields and public getter setter methods.

Benefits

- 1) data hiding : Prevents direct modifications of variables
- 2) control over data : validation logic can be applied inside setters

3) Better maintainability

4) Improved security

5) Loose coupling:

class Card

private String cardNum;

public getCardNum

public setCardNum
validate

8) Abstraction :

- (i) Abstraction is the process of exposing only essential functionality of an object while hiding its implementation details. typically achieved using abstract class or interfaces.

etc

```
interface CardPaymentService {
    void initiateCardPayment()
    void validateCardDetails()
}
```

```
class VisaCardService implements Card
class VisaCardPaymentService implements CardPaymentService {
    public void initiateCardPayment() {}
    public void validateCardDetails() {}
}
```

```
VisaCardPaymentService paymentService =
    new CardPaymentService();
paymentService.validateCardDetails();
```


4) Interface vs Abstract class. IMP

A) Interface 3

- ① Interface is a contract, it specifies what a class can do but not how it is done.
- ② Implementation is (implemented) provided by implementing classes.
- ③ Supports multiple inheritance that means a class can implement multiple interfaces.
- ④ used for abstraction + polymorphism.
- ⑤ widely used in springboot (api) → service + serviceimpl

↓
waha a service → its implementation can do

```
interface CardPaymentService {
    void validateCardDetails();
    void initiateCardPayment();
}
```

```
class VisaCardPaymentService {
    void validateCardDetails() {
        // ...
    }
    void initiateCardPayment() {
        // ...
    }
}
```

B) Abstract class :

- ① is partially implemented class. Has
 - ② Abstract methods (without body)
 - ③ Concrete methods (with body)
 - ④ Instance variables & constructors
 - ⑤ used in where class share common state &/or behaviour.
 - ⑥ Does not support multiple inheritance ...
- ~~two common method in p...~~

```
abstract class CardPayment {
```

```
    private String CardNumber;
```

```
    Card (String CardNumber() {
```

```
        this.CardNumber = CardNumber;
```

```
    abstract void initiateCardPayment();
```

```
    void getCardDetails() {
```

```
        sout(CardNum);
```

```
    }
```

So interface cannot have ~~in~~ constructor, instance variable, it can only have public static final variable is constant and abstract/default/static methods. ^{can be overridden} ^{cannot}

whereas abstract class can have constructors, instance variables, both default and concrete abstract methods.

5) Can constructors be polymorphic

No,

① Constructor cannot be polymorphic because constructors are not inherited and not be overridden. Constructors invoked at object creation step, and resolved at compile time stage not at runtime stage.

② Polymorphism, (especially runtime polymorphism) which is achieved at runtime by method overriding, where JVM decides at runtime which method implementation to call based on object.

③ Since constructors :

- ① are not inherited
- ② cannot be overridden
- ③ called during object creation
- ④ resolved at compile time

⑤ Constructors are executed in fixed order → constructor chaining
note. constructors supports overloading not overriding.

6) Dynamic Method Dispatch

① is a mechanism in java where jvm decides at runtime which method implementation to execute during runtime not compile time based on object type not reference type.

```
Card c1 = new VisaCard();
c1.pay();
```

↳ runtime - run pay() of VisaCard

Card: Parent class, reference type

VisaCard: Object time, subclass.

* runtime polymorphism.

- jvm
- equ
- even
- inst
- alr
- un
- loc
- even

Q # 1) What is Singleton class?

① A singleton class is a class that allows only one object (instance) to be created throughout the application and provides global access point to that instance.

② It is used for:

- a) shared resources (configuration, logging, cache)
- b) Database connection manager
- c) Application-wide web services.

Q 2) How to create singleton class?

- ① Make constructor private: prevents object creation using new()
- ② create static ^{instance} variable which holds single object
- ③ provide public static method which returns same instance everytime.

class Singleton {

private static Singleton ^{instance} singleton;

private Singleton() {}

public static ^{synchronized} Singleton getInstance() {

if (instance == null) {

instance = new Singleton();

}

return instance;

}

- JVM must acquire lock
- even when instance is already created
- unnecessary locking on every call

- ① Every time a thread calls `getInstance()`
- ② JVM must acquire a lock
- ③ Even when instance is already created
- ④ unnecessary locking happens at every ~~etc~~ call.

Note: All Spring beans are singleton by default

2) Thread Safe Singleton with Double check locking?
Is singleton thread safe.

```
class Singleton {
```

```
    private static volatile Singleton instance;
```

```
    private Singleton() {}
```

```
    public static Singleton getInstance() {
```

```
        if (instance == null) {
```

```
            synchronized (Singleton.class) {
```

```
                if (instance == null) {
```

```
                    instance = new Singleton();
```

```
                }
```

```
            }
```

```
        }
```

```
        return instance;
```

```
    }
```

```
}
```

1) Thread 1 enters $\rightarrow$ calls `getInstance()`

`instance == null` $\rightarrow$ true

go enters synchronized block

check again `instance == true`

creates instance $\therefore$ stores

new singleton object exist

2) Thread 2 calls `getInstance()`

`instance == null` $\rightarrow$ False

skips ~~sa~~ synchronized block

returns existing object.

$\therefore$ faster $\rightarrow$ no locking

c) Two threads arrive at same time

T1 and T2 reach in first `instance == null` if block

$\rightarrow$ true

T1 first enters synchronized block

T2 waits

T1 creates instance (object)

arrives late

T2 enters synchronized block

second `if (instance == null)` false

$\rightarrow$ does not create new object

① A synchronized `getInstance()` method is thread-safe but introduces synchronization overhead on every call whereas, double-checked locking synchronizes only during first instance creation, improving performance.

② volatile keyword ensures visibility changes across threads and prevents instruction reordering.

Guaranteeing that singleton instance is fully initialized before any thread access it.

* Collections

1) What is Java Collection framework?

Ans

① Java collection framework is a unified architecture that provides interfaces for commonly used data structures like List, Set, Queue, Map along with their implementation classes and utility algorithms to effectively store, ^{manage} and manipulate collection of objects.

② JCF provides

① Standardize APIs

② Optimised performance

③ Reusable implementation

④ Easy data manipulation (Sorting, searching)

⑤ Before JCF, developers had to implement their own data structures

2) Enhancement in Java 8 in collections ?

Ans : Java 8 introduced several important improvements mainly to support functional ~~for~~ programming and parallel stream.

A] Addition of Stream API :

- ① Collections can be converted to streams to perform functional ~~style~~ ^{style} operations.
- ② Operations like `map()`, `filter()`, `collect()`;
- ③ Enables parallel processing using `parallelStream()`

B] Addition of new default methods in Collection interface

- ① `forEach()`
- ② `removeIf()`
- ③ `splitterator()` → iteration + divide in chunks for parallel traversal

C] Map interface enhancements

- ① `forEach()`
- ② `compute()` `computeIfPresent()` `computeIfAbsent()`
- ③ `merge()`
- ④ `replaceAll()`
- ⑤ `getOrDefault()`

3) Iterator vs ListIterator.

Ans: Iterator is used to traverse any collection in forward direction and supports only remove operation.
ListIterator works only with List implementations supports both forward and ~~reverse~~ backward traversal add, set and remove operations.

4) Common methods in Collection interface

add()

remove()

addAll()

removeAll()

size()

stream()

toArray()

iterator()

5) Algorithm in Array.sort() vs Collection.sort()

Ans) A) Array → a) primitive types int char double

→ Dual Pivot quicksort

b) Objects

→ TimSort → hybrid of merge + insertion sort

B) Collections → TimSort

List Interface

- List is an interface in java collection framework that represents ordered collection allowing duplicate elements and index based access.
- Common implementation include ArrayList, LinkedList, Vector, Stack.

6) ArrayList vs LinkedList (Internal working)

A) ArrayList

- 1) uses dynamic array
- 2) When capacity is full, new larger 2x size array is created and all elements are copied.
- 3) Provides fast random access $O(1)$ because elements are stored contiguously.
- 4) Used when frequent access/read operations index based access
- 5) ^{used when} Insertion mostly at end because insertion in middle will require to shift all the elements.

B) LinkedList

- 1) Used Doubly Linked List
- 2) Each node stores
 - a) ~~data~~ data
 - b) reference to previous node
 - c) reference to next node.
- 3) Insertion time $\rightarrow$ no shifting required
- 4) Accessing takes $O(n)$

Frequent

- (4) Used when insertion/deletion in middle or end only change of reference/pointer required no shifting
- (5) Less random access

Set Interface

- Interface in java collection framework that represent collection of unique elements where duplicate are not allowed
- Common implementations are:
HashSet, LinkedHashSet, TreeSet

HashSet : no ordering fast performance

LinkedHashSet : maintains insertion order

TreeSet : maintains sorted order

7) hashCode() equals() contract. why override

Ans

(1) equals() is a method of Object class which compares two objects for logical equality (whether two objects have same content).

(2) hashCode() is a method of Object class which returns a integer hash-value to determine which is used by hash based collection like HashSet / Hash map to determine the bucket/location for storing objects.

contract :

② If two objects are equal by equals() method they must return same hashCode.

④ Since Object class is a parent of all classes, all objects have these two methods equals() hashCode().

When equals() method is overridden, hashCode() method must also be overridden so to maintain correct behaviour in hash based collections.

③ If two ~~methods~~ objects have same hashCode(), they may or may not have be equal().

... collision.

```
public class Card {
```

```
    private int id;
```

```
    private String cardName;
```

```
    public Card(int id, String cardName) {
```

```
        this.id = id;
```

```
        this.cardName = cardName;
```

```
    }
```

```
    @Override
```

```
    public boolean equals (Object obj) {
```

```
        if (this == obj) return true;
```

```
        if (obj == null || this.getClass() != obj.getClass()) return false;
```

```
        Card other = (Card) obj;
```

```
        return this.id == other.id &&
```

```
            Objects.equal(other, cardName, this cardName);
```

```
    }
```

@ Override.

```
public int hashCode() {
    return Objects.hash(id, cardNumber);
}
```

Q) How HashSet handles duplicate Internal working
Ans OR HashMap no

① Internally use array of bucket.

① When an element is inserted in HashSet, its hashCode() is calculate to determine the bucket location.

② If the bucket is empty, element is inserted directly.

③ If another element already exist in the same bucket, equals() method is used to compare the new object with existing object.

④ If equals() method returns true, the new object is considered as duplicate and not inserted.

⑤ If equals() method returns false, collision occurs and new object is then inserted in same bucket in a linkedlist which is at the same bucket where collision occurred.

⑥ If linkedlist grows beyond certain threshold, in java 8+ it is converted to balanced tree (TreeNode, Red Black Tree) for better performance.

HashMap / HashSet (Retrieval get operation)



- ① hash(key)
- ② bucket index
- ③ traverse LL/Tree and use equals() to get the key

a) Linked HashSet

- ① Set implementation maintains insertion order while not allowing duplicates.
- ② uses ~~doubly~~ Each node/entry has two pointers
 - before $\rightarrow$ points to previous element
 - after $\rightarrow$ points to next elementforms doubly linked list maintains insertion order.

Entry $\Rightarrow$ hash
key (data)
next (for bucket chain)
before
after

Note: ~~for~~ collision inside bucket handled using linkedlist/tree

* Linked HashSet uses hash table for fast lookup and each entry additionally maintains before and after pointers forming a doubly linkedlist which preserves insertion order.

TreeSet

- ① Implementation of Set that stores unique element in sorted order using Red-Black-Tree.
providing $O(\log n)$ time complexity for add, remove, search operations

10) HashSet LinkedHashSet TreeSet use cases?

Ans

- ① HashSet is used when only uniqueness and fast performance is required.
- ② Linked Hash ^{set} ~~map~~ is used when uniqueness with insertion order is needed.
- ③ TreeSet uniqueness with sorted order.

11) HashMap internal working

Ans

- ① Array of bucket is used.
- ② When a new key is inserted, `hashCode()` is calculated to determine the index of bucket.
- ③ If bucket is empty, create node & insert.
- ④ If bucket already has an entry $\rightarrow$ collision
 - * - use `equal()` method to compare new and existing.
 - if key already exists update value.
 - if key is different add node to bucket (LL/Tree)

12) Java 8 improvements HashMap

- ① In java 8 hashmap was improved by converting high collision buckets to from LinkedList to Red Black Tree when entries exceed 8, improving worst case performance to $O(\log n)$.
- ② Better hash algorithm to reduce collision.
- ③ If bucket size reduces below 6, tree converts back to LL to save mem.

13) ~~ArraySet~~ vs Hashmap vs Treemap

Ans

- ① Hashmap stores key-value pairs without maintaining order and provides $O(1)$ avg performance.
- ② Treemap stores key in sorted order using RedBlackTree and provides $O(\log n)$.

∴ can we use objects / classes as key in Map

Ans ... Yes if with certain rules

- ① override equals() hashCode()
- ② Equal objects ~~shd~~ must create same hashCode()
- ③ hashCode() → bucket location equals() → logical comparison
- ④ Keys should ideally immutable
if fields used in equals/hashCode ~~change~~ change after insertion, map may fail to retrieve the value
- ⑤ Strings are commonly used as keys because immutable & well-defined equals() hashCode() implementations.

15) Choosing right collection

Ans A] When ordering is NOT important

- ① HashSet → unique elements, fastest performance
- ② HashMap → key value pair, fast lookup

B] ordering imp Insertion

- ① LinkedHashSet
- ② LinkedHashMap

c) Sorted order

① TreeSet

② TreeMap

d) duplicates are allowed

① ArrayList → fast random access, (reading)

② LinkedList → frequent insertion/deletion

e) Multithreaded env

① Concurrent HashMap

② CopyOnWriteArrayList

3 methods()

- hasNext()

- next()

- remove()

interface in java collections used to cycle through a collection one element at time

- Fail-Fast ~~iteration~~ iterator throws ConcurrentModificationException if collection is structurally modified during iteration

- maintains a modCount field ... if modified ... exp

- eg ArrayList, HashMap, HashSet, LinkedList

- Fail-Safe iterator works on ~~a~~ clone/copy of collection so modification does not affect iteration.

- eg CopyOnWriteArrayList, ConcurrentHashMap

Multi threading



1) Process is an instance of a program that is being executed. When a program runs OS creates a process to manage its execution.

2) A thread is a smallest unit of execution within a process. A process can have multiple threads, which share same resources but can run independently.

3) Multitasking: OS runs multiple process simultaneously
1 core $\Rightarrow$ rapidly swithching between tasks
multi core $\Rightarrow$ true parallel execution happens

4) Multithreading = ability to run multiple threads of a single program simultaneously / concurrently.

time slicing: CPU divides time into short intervals, allocating each process a slice.

context switching: CPU pauses running task (process/thread), saves (PCB) _{control block}, and loads the saved state of new process.

Thread t = new Thread () $\rightarrow$ { };

1) Creating thread using Thread class

```
class MyThread extends Thread {
    public void run() {
        sout("Thread running");
    }
}
```

psvm {

MyThread t1 = new MyThread();

t1.start();

}

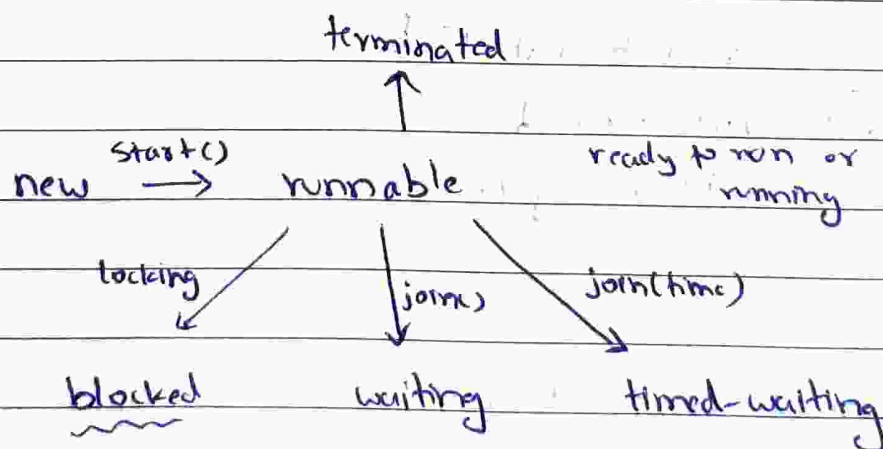
provisions necessary memory

(Int@)

creates and launches new separate thread of execution (OS)

run() → in new call stack

* this approach is not preferred because java supports single inheritance and it mixes thread management with business logic.



if a thread tries to enter synchronized block and a lock is already held, it goes into BLOCKED state.

implementing runnable interface.

```
public MyClass implements Runnable {
    @Override
    public void run() {
        sout("running");
    }
}
```

```
psvm {
    MyClass myclass = new MyClass();
    Thread t1 = new Thread(myclass);
    t1.start();
}
```

OR

```
Runnable t = new Runnable() {
    @Override
    public void run() {
        sout("running");
    }
}
```

```
new Thread(t).start();
```

OR

```
Runnable t = () -> sout("");
new Thread(t).start();
```

Ques why runnable preferred over Thread?

+ Runnable is preferred because

- separates task from thread
- allows class to extend another class

while Thread

- ↳ java supports single inheritance
- ↳ mixes thread management & business logic

1) What is Runnable interface and why is it preferred?

Ans:

① Runnable interface is a functional interface in java, i.e. it has only one abstract method `run()`, which is used to provide a task that has to be executed by a thread.

② Basically it represents a task that has to be executed by thread.

③ Either we implement the interface or directly use lambda to provide implementation.

Why Preferred:

① It separates the thread from task execution, and allows class to extend another class since java does not support multiple inheritance via classes.

② If we extend Thread class it cannot extend any other class, because java does not support multiple inheritance via classes. So by implementing Runnable our class still can extend another class and define task that has to be executed in thread in `run()` method.

then we pass Runnable object to thread. → Thread.start()
eg -

class A

class B extends A ✓

extends A and Thread X

extends A, Thread X

extends Thread ✓ but not A

→ class B extends A implements Runnable {

@Override

void run() {

}

}

PSVM

Thread t = new Thread(b)

B b = new B()

t.start();

③ Also, Runnable is extensively used in modern multithreading framework :- Executor Service.

↳

How it is used in your project

↳ kafka producer/consumer

2) Thread Safety ⇒ means ensuring shared resource is accessed correctly by multiple threads running concurrent. Prev 3 →

lock ⇒ synchronized, lock reentrant, atomic class, concurrent collection, immutable objects
AtomicInteger

Synchronized keyword



- ① is used for thread synchronization, by allowing only one thread to access/execute a critical section of code at one time.
- ② Critical section of a code is a part of a code that accesses shared resource and therefore must not be executed by multiple thread at a same time.
- ③ It prevents race condition, data inconsistency, data loss.
↳ occurs when multiple threads access/modify shared resource at same time.
- ④ types:
Synchronized method, block, static method.

So how it works internally is ↳ used for sync. allows one thread

- ① In java every object has a intrinsic lock/monitor lock.
When a object is created, jvm attaches this to it.
- ② So when a thread '1' enters a synchronization block.
 - a) it acquires object's monitor lock.
 - b) other thread '2' trying to enter this synchronization block must wait.
 - c) when thread '1' finishes execution, it releases the lock.
 - d) Now, the waiting thread 2 acquires lock and executes synchronized block.
- ③ (class level lock & object level lock)

5) wait() notify() notifyAll()

Ans : (1) are from Object class and are used for inter-thread communication.



(2) must be used in synchronized block/method.

(3) allow thres to coordinate their execution when working with shared resource.

a) wait() => causes the current thread to release lock and wait

b) notify() => wakes one waiting thread()

c) notifyAll() => wakes all waiting thread()

Why needed => Sometimes thread must wait for a condition before continuing.

How they work.

(1) Thread '1' enter a synchronized block and acquires lock

(2) Thread 'i' calls wait()

(3) Thread '1' releases the lock and goes to waiting state.

(4) Another thread '2' acquires lock, enters synchronized block and updates the condition.

(5) That thread '2' calls notify() or notifyAll()

(6) Waiting thread wakes up and require lock before continuing.

Ex: Producer Thread = produces data

Consumer thread = consumes data

If producer is no longer ~~consumin~~ producing,

consumer runs when data is not available, it should

wait instead of continuously checking. busy -> cpu waste

6) Producer Consumer Problem



- ① is classic multithreading problem where one thread produces data and other thread consumes data while sharing common buffer.
- ② The main challenge is that, producer thread should not produce datum when buffer is full and consumer thread should not consume data if buffer is empty.
- ③ In java, this co-ordination can be implemented using synchronization along with `wait()` & `notify()` methods for inter-thread communication.
- ④ These methods allow thread to wait for a condition and be notified when condition changes.
- ⑤ In modern java applications, higher-level utilities like `BlockingQueue` are commonly used.

```
class Buffer
```

```
    private int data;
```

```
    private boolean available = false;
```

```
    public synchronized void produce(int value) throws InterruptedException {
        while (!available) {
            wait(); // wait if buffer is full
        }
```

```
        data = value
```

```
        System.out.println("produced " + value)
```

```
        available = true
```

```
        notify();
```

```
    }
```

synchronized

```
public void consume() throws InterruptedException {
    while (!available) {
        wait(); // if no data wait.. dont consume.
    }
```

```
    save ("consumed " + data)
```

```
    available = false;
```

```
    notify();
```

```
    return data;
```

```
}
```

produce -> add element in queue if queue full block

consumer ->

produce -> adds elem

not full
not empty

1) acquires lock

2) check if queue is full

if full -> not full wait

3) insert element

4) ~~not~~ signal not empty

5) release lock

psvm {

Thread producer

Buffer buffer = new Buffer();

Thread producer = new Thread() -> {

try {

for (i = 0; i <= 5; i++) {

buffer.produce(i);

}

catch (Exception) {}

}

}

Thread consumer = new Thread() -> {

try {

for i = 1 to 5

buffer.consume();

}
catch Excp

}

}

producer.start();

consumer.start();

Executor Service + blocking queue

- ① Thread safe queue implementation that blocks thread when queue cannot be used immediately.
 (thread automatically wait when queue is full/empty)

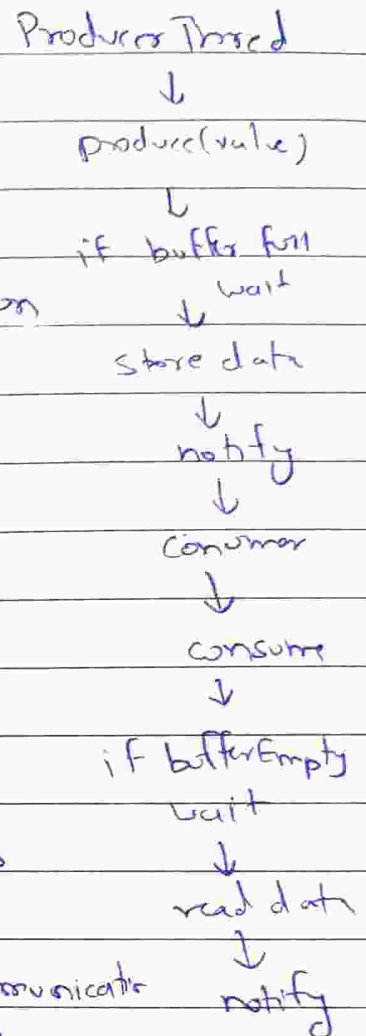


② Producer tried to add → queue is full → producer waits()

③ Consumer tries to consume → queue is empty consumer waits()

④ thread automatically waits until operation becomes possible

⑤ implementation :
 Array Blocking queue
 Linked Blocking Queue
 Priority Blocking Queue
 Delay queue
 Synchronous queue.



⑥ It uses Reentrant lock and condition variables

- a) provides explicit locking (w/o synchronization)
- b) condition allows to perform inter-thread communication and inter-thread coordination → can be used when desired state change.

two conditions

not Full → lock.newCondition()

not Empty → lock.newCondition()

has put() take()
 waits if queue is full if a is empty

Producer

lock.lock()

if queue full

not Full.wait()

insert element

not Empty.signal()

lock.unlock()

Consumer

lock.lock()

if queue is Empty

not Empty wait

consume data

not Full.signal()

lock.unlock()

7) Reentrant lock

Ans: (1) is an implementation of lock interface

for controlling access to shared resources in multithreading

- (2) it provides explicit locking, programmers manually acquires and releases the lock
- (3) Reentrant meaning, a thread that already holds a lock can acquire it again without getting block.
- (4) It is needed because it provides more control and flexibility over synchronized locking.

How it works

- (1) Create Reentrant lock
- (2) call lock() to acquire the lock
- (3) Execute critical section
- (4) Call unlock() to release the lock.

```
class Counter {
    int count = 0;
    ReentrantLock lock = new ReentrantLock();
```

```
    void increment() {
        lock.lock()           // acquire lock
        try {
            count++;
        }
        finally {
            lock.unlock();    // release lock
        }
    }
}
```

① `tryLock()` $\Rightarrow$ try to acquire lock without waiting
 ↳ tries to acquire lock immediately
 - if lock is acquired by another thread it returns false instead of waiting
 * to skip

② ~~lockInterrupt~~ `lockInterruptibly()`;
 ↳ Allow a thread waiting for lock to be interrupted.

③ Fairness Policy:
 - enforce fair scheduling of threads.
 - with fairness enabled threads get the lock in order of waiting. (FIFO)

④ `ReentrantLock lock = new ReentrantLock(true);`
 = enables it

⑤ T1 waits T2 waits T3 waits
 lock released
 T1 gets first T2 next T3 next

⑥ `lock.newCondition()`

`condition.await()` // similar to `wait()`

`condition.signal()` // similar to `notify()`

" // only one wait queue possible

we can have different queues for different conditions

not full } producer
 not empty } consumer

T1 } same queue
 T2 }
 T3 } different condition

Q) CAS

instruction executes as
single, indivisible unit
no other thread sees it

low level cpu



Ans: ① is a low level Atomic operation, used in
operations happens
completely or not at all

concurrent programming

- ② How it works is .. it updates a variable only if current value (ie value in memory) matches expected value or else fails and retries with new value.
- ③ it works by comparing the current value in memory with an expected value and update it if they match, otherwise operation fails and thread retries.
- ④ No lock and synchronization is required
- ⑤ CAS faster because
 - a) no thread blocking
 - b) no context switching $\Rightarrow$ saving state of one process/thread and loading the state of another thread where left
 - c) lock free algorithm
 - d) uses cpu atomic instructions

eg value in memory (V)
Expected value (A)
new value (B)

if (V == A)

V = B

else

fail $\rightarrow$ retry

when atomic over synchronization/locks

preferred when we need thread safe update for single variable without overhead of locks.



a) Atomic classes:

Atomic classes are thread safe classes provided by package `java.util.concurrent.atomic`

allow atomic operations and provide thread safety on data without using locks and synchronized blocks.

It uses CAS operations, to update value only if expected value matches the current value in memory.

If update fails due to concurrent modification the operation retries until it succeeds.

lock-free thread safety

eg - API counter

better performance

rate limiter counter

a) avoids thread blocking

metric

b) no context switching

retry attempt

c) lock free

d) suitable for simple shared variables

```
class Counter {
```

```
    Atomic Integer count = new Atomic Integer(0);
```

```
    void increment() {
```

```
        count.incrementAndGet();
```

```
    }
```

```
}
```

↳

set
get

incrementAndSet

getAndIncrement

Atomic Long

Boolean → compareAndSet

Reference →

used for object ref

StampedReference → ABA problem with version

1.9) ConcurrentHashMap



- ① Before it lets explain normal Hashmap & CAS
- ② is a thread safe implementation of Hashmap
- ③ It allows multiple threads to read and update the map concurrently without locking the entire map.
- ④ Normal Hashmap $\rightarrow$ when multiple thread result in inconsistent data, infinite loop during resize, lost updates

How it works internally is : ^{volatile}

- ① uses array of bucket + LL and RBTee when collisions.
- ② lock-free reads : multiple threads can perform `get()` op or read the data without need of acquiring a lock.
- ③ Instead of locking the entire map, only the bucket being modified is locked.
so this allows ^{different} multiple threads to update/modify different buckets simultaneously.
- ④ CAS atomic operation is used to perform insertion
 $\hookrightarrow$ `put()` if CAS fails $\rightarrow$ synchronize bucket
 - ① compute hashcode
 - ② determine/identify bucket index
 - ③ if bucket empty $\rightarrow$ CAS insertion
 - ④ if bucket is occupied ^(collision) $\rightarrow$ synchronize on bucket
 - ⑤ Insert into LL or RBT
 - ⑥ Resize table if threshold reached

eg → in memory caching
api request counting, login attempts
rate limiting systems

↳ controls how many requests a client/user
can make to a service within a specific time
period.

- prevents server overload
- API abuse
- DDOS

alog = token bucket, leaky bucket
↳ only if token present ↳ request allowed at constant rate

ii) Challenges in multithreading

Ans: (1) Race condition: when multiple thread modify shared data w/o proper synchronization

(2) Deadlock: one or more thread wait for each others lock indefinitely.

(3) Thread starvation: one thread does not get CPU or lock because other thread keeps acquiring resources

(4) Visibility issues: Due to CPU caching and memory reordering updated data by one thread may not be visible to other threads

10) Exchanger

- (1) is a synchronization utility from java.util.concurrent that allows ~~the~~ two threads to exchange data at synchronization point
- (2) Each data call exchange() method and blocks until another thread arrives with its data.
- (3) When both thread reaches exchange point, the objects are swapped and each thread receives other threads data.
- (4) useful for coordinating btw thread for hand off data without using shared resource.

```
Exchanger<String> exchanger = new Exchanger<>();
```

```
new Thread() -> {
```

```
try {
```

```
String data = "A" receives
```

```
data = exchanger.exchange(data)
```

```
out(data)
```

```
} catch (Exception e) {
```

```
} .start();
```

```
} new Thread() -> {
```

```
try {
```

```
String data = "B"
```

T1 receives B

T2 receives A

eg Buffer swapping betⁿ producer / consumer

③ Integrates with service registry and does lb for multiple instances

Multi threading

① What is a thread?

Ans ① A thread is a smallest unit of a process

② Process is an instance of a ~~go~~ program.

③ Process can have multiple threads that can run concurrently, share same resources and run independently.

④ All threads in a process share same heap memory but have their own stack memory.

⑤ Threads allow program to run multiple tasks concurrently.

⑥ In java we can create threads 3 ways:

a) a Thread class

b) Runnable interface

c) Callable interface (with executor framework)

2) Multitasking

① allows os to run multiple process simultaneously.

② on single core cpu it is done by time sharing and rapidly switching between tasks.

③ os scheduling, time slicing $\Rightarrow$ divides $\neq$ cpu time into intervals.

3) Multithreading

Ans: ① means to run multiple threads concurrently within a process

② so it is ability of a program to run multiple threads concurrently to perform multiple tasks at same time within a process.

3) Java Memory model, visibility problem and volatile.

Ans:

① Defines how thread interacts with shared memory and ensures visibility and ordering of variables updates between threads

② Variables are stored in main memory

Thread copy variables to its own CPU cache.

Thread works on its local cached copy

Changes may not be immediately update to main memory

resulting visibility issues where other thread working

on same shared variable still read old value.

How JVM solves this visibility issue:

A) Synchronized

operation can happen completely or not at all

* one thread at a time + no other thread

① ensures atomicity and visibility

② how -> when thread enters synchronized block it refreshes data from main memory

-> when thread exits synchronized block it writes data changes to memory.

13] volatile :

context : In multithreaded systems, each thread may use its own CPU cache instead of directly ~~the~~ reading from main memory, which can cause visibility issues where one thread cannot see updates made by other thread.

① It ensures visibility ~~change~~ of shared resource across threads.

② When a variable is declared using volatile keyword, read and write operations on it are directly done in main memory, ~~as~~ ensuring all threads see the latest value.

③ does not guarantee atomicity for compound operations like increment

④ typically used for flags

4) Executor Framework

A) Problem with ~~existing~~ Manual Thread

① expensive to create, requires OS and memory/cpu.

eg. for 1000 requests, each request creates a new thread.

② cant reuse thread, once thread done executing the task it dies. now for new task new thread has to be created again

③ cant control no. of thread.

too many threads get created causing high cpu mem

④ no task management, no queue, no scheduling.

So with Executor Framework:

① Our developers just have to submit task. not threads

② ExF manages

- a) task submission

- b) thread creation

- c) thread reuse

- d) task queue

- e) thread lifecycle.

Thread pool is a collection of reusable worker threads that execute tasks.

∴ Instead of creating new threads every time, tasks are submitted to a pool.

14) Executor Framework uses thread pool to manage fixed number of reusable worker threads.

1) Task are submitted to executor and placed in a queue, available thread picks task from the queue for execution.

2) Performance is improved by reusing threads and controlling number of ^{current} threads.

1.1) Executor interface:

1) provide simple way to execute tasks asynchronously without manually managing threads.

2) only provides execute() method for running a Runnable tasks.

3) basic => accepts runnable, no return value, no exception tracking.

1.2) ExecutorService:

1) Sub-interface of Executor provides advance thread-pool management and task lifecycle control.

2) most commonly used.

3) has => Thread pool management

Task result handling

Task cancellation

graceful shutdown.

ExecutorService executor = Executors.new^{fixed}ThreadPool(3)

↳ Task → Queue → worker threads

④ has a `submit()` method

- submits a task and returns Future object to track result
- takes callable or runnable

```
ExecutorService executor = Executors.newFixedThreadPool(3);
```

```
Future<Integer> future = executor.submit(() -> {
    return 10 + 30;
});
System.out.println(future.get());
```

① <code>future.get()</code>	get task result
<code>future.isDone()</code>	check if task completed
<code>future.cancel()</code>	cancel task

⑤ `executor.shutdown()` (F) ~~(D)~~ `executor.invokeAny`
`invokeAll`

↳ gracefully stops executor service

- stops accepting new task
- finish already submitted task
- then terminate threads

`executor.shutdownNow()`

↳ stops immediately
attempts to interrupt running tasks
returns a list of waiting tasks

A) executor.invokeAll()

- ① Execute & collect of tasks and wait until ^{all} task is finish
- ② Returns a list of future objects

```
ExecutorService executor = Executors.newFixedThreadPool(3);
```

```
List<Callable<Integer>> tasks = List.of(
```

```
    () -> 1+1,
```

```
    () -> 2+2,
```

```
    () -> 3+3
```

```
);
```

```
List<Future<Integer>> results = executor.invokeAll(tasks);
```

```
for (Future<Integer> future : results) {
```

```
    out(future.get());
```

```
}
```

```
executor.shutdown();
```

B) executor.invokeAny()

- ① executes multiple tasks but returns result of first task that ~~exp~~ completes successfully.
- ② stops waiting after one task is finished.
- ③ other tasks are cancelled
- ④ returns single result ~~future~~ object

Runnable | Callable | Future

- ① Both represent a task that will be executed by thread concurrently
- ② Runnable func interface → defines a ~~call~~ run() method does not return any result
- ③ Callable similar → defines a call() method that can return a value and throw checked exceptions.

execute() → only takes runnable

submit() → takes both runnable and callable.

↳ returns a Future Object used to retrieve result of Callable task.

Future Object :- represents result of async operation that will be available in future.
- acts as a placeholder for the task completion and result available later.

* When a task is submitted to ExecutorService using submit() it returns a Future Obj immediately while task executes in ~~sp~~ separate thread.

→ retrieve result using get() → also throws exception if fails
check task completion isDone()

CallableTemplate whenComplete()

Thread pool implementations

- 1) FixedThreadPool → fixed no of thread -- queue
- 2) CachedThreadPool → dynamically created threads and reuses the short lived task
- 3) Single Thread Executor → one thread execute task subsequently
- 3) ScheduledThreadPool → schedule task run after delay periodically.

Thread pool Executor

- ① ~~Also~~ Core implementation in java allows us to manage thread pool creation using parameters like corePoolSize, maximumPoolSize, task queue size. (task wait when all cores are busy)
- ② Task first use core threads, wait in a queue, new threads are created up to max pool size, if needed.
- ③

Thread Pool Task Executor

- ① In Spring we have TPTE bean ^{added in classpath} and configure, with spring-boot-starter web dependency.
- ② Provide own configuration using properties like
 spring.task.execution.pool-core-size =
 pool-max-size =
 queue-capacity =
- ③ In main Application class we do
 @EnableAsync.
- ④ Now any method annotated with @Async will run in Thread Pool Executor thread and runs asynchronously w/o blocking calling method.

③ we can also provide manual configuration by creating our own @Bean @Configuration

@Bean

```
public ThreadPoolTaskExecutor taskExecutor() {
```

```
    ThreadPoolTaskExecutor executor = new TPTE();
```

```
    ex.setCorePoolSize(5)
```

```
        MaxPoolSize(10)
```

```
        QueueCapacity(50)
```

```
    ex.initialize();
```

```
    return executor;
```

```
}
```

Future: asynchronous operation by another thread
not main thread

placeholder value (immediately) → allows main thread
to continue

* after a while when you do
future.get() if task completed you will get the value
BUT, if task not completed → blocking

perform asynchronous computation and trigger dependent computations which could also be asynchronous.



Completable Future :

- ① represents asynchronous, non-blocking operation whose result will be available in future.
- ② Its non-blocking execution
supports chaining
has better exception handling
- ③ what happens is task executes asynchronously and callbacks are attached.

eg `= kafkaTemplate.send("order-topic", orderId, data),`
`CompletableFuture<SendResult<String, OrderResponse>> future =`

```
future.whenComplete((result, ex) -> {  
    if (ex != null) handle  
    else log  
})
```

eg ① `supplyAsync()`

- ② runs task in separate thread, runs asynchronously that ~~resul~~ returns a result.
- ③ results can be later process using callbacks

`CompletableFuture<String>.supplyAsync(future = completableFuture.supplyAsync()`
`(-> restCall.getUserId())`

Starts async
rest call

CompletableFuture <Response> future =

CompletableFuture.supplyAsync(() -> userService.(restcall))

- thenApply(transform result)
- or • thenAccept(consume result (no return))
- exceptionally()

Response res = future.join();

// give result or wait

Executor Framework

- ① High level API to manage and control thread execution
- ② we don't have manually create, manage, thread using thread class
- ③ Executor Framework takes care of thread pool creation, managing its lifecycle, thread reuse, task queue.
- ④ We just have to submit a task that has to be executed concurrently in separate thread

(A)

4 Executor basic interface => has execute() -> fire and forget
no result

+ ExecutorService : ① creation thread pool -> newFixedThreadPool(n)
② get result newCachedThreadPool()
③ shutdown threadpool newSingleThreadExecutor()
newScheduledThreadPool()



+ `execute()`

- takes runnable
- no result
- no excep handling

```
executor.execute(() -> { } );
```

+ `submit()`

- takes runnable, callable
- Returns Future
- can track result & excep

```
Future<Obj> future = executor.submit(() -> { } );
```

⑦

- Future represents asynchronous computation, value will be available in future

- has methods like `get()` ... blocks
`isDone()` ... to track result
`cancel()` ... cancel task.

- Problem with future is - `get()` is blocking
- not callables / chaining

completable future = `CompletableFuture.supplyAsync(() -> {}, executor);`

transformation

`future.thenApply(data -> data ... )`

consume
result &
returning

• `thenAccept(result -> log);`

• `exceptionally(e -> {})`

• `whenComplete(?, (result, ex))`

Completable Future (with example)

Recap: `Future.get()` → if result ready fine
 if result read main thread block
 * not suitable if we have a loop / list of tasks and want to perform dependent operations on it.

for (i to 100)

`CompletableFuture.supplyAsync()` → `getOrder()`

- `thenApply (order → enrich (order))`
- `thenApply (o → performPayment(o))`
- `thenApply (order → dispatch (order))`
- `thenAccept (order → sendEmail (order))`

① result of `supplyAsync` will be the parameter of `thenApply` lambda → result is passed as a input

② `thenApply` lambda will also be performed by same thread which performed `supplyAsync`

③ `thenAccept` takes input ... Returns nothing
`thenApply` takes input ... returns output

↳ another `CompletableFuture` return type:

↳ `getAsync`

2. overloaded implementations

① `supplyAsync (Supplier <T> supplier)` ⇒ uses `ForkJoinPool.commonPool()`
 ie task

② `(Supplier <T> supplier, Executor executor)`
 ↳ for IO

Executor
Executor

cpuExecutor = Executor.newFixedThreadPool(4);
ioExecutor = Executor.newCachedThreadPool();



own threadpool
executor

CompletableFuture.supplyAsync(o -> get order(), cpuExecutor)

• thenApplyAsync(o -> enrichOrder(o), ioExecutor)

• thenApply(o -> performPayment(o))

• exceptionally(e -> new FailedOrder())

• thenApply()

• thenAccept()

any exception thrown
by above 3 callbacks

by default

- or else uses ForkJoinPool.commonPool()

- Java's default thread pool

- shared globally across JVM

- designed for parallel CPU bound work

@Async in Spring Boot

- If a method is annotated with @Async that will run in separate thread. - if executor not provided it will use spring own SimpleAsyncTaskExecutor

- Basic Basically creates a new thread per task -> risky

- Better to have own threadPool Bean

@Bean("ioExecutor")

public Executor taskExecutor() {

ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();

executor.setCorePoolSize(5); // keeps atleast 5 thread alive idle

setMaxPoolSize(10); // up to 10 allowed

holds upto 100 pending task

setQueueCapacity(100);

before creating extra thread

setThreadNamePrefix("async-")

• initialize()

return executor;

now Spring looks for

- ① A bean named taskExecutor
- ② or any "Executor" bean if only one exists.
- ③ or specifically named executor via @Async("beanName")

IMPORTANT

- ① use @Async => to run something in background
fire & forget
- ② use CompletableFuture => when you need to
combine, chain, or co-ordinate
async task

makes a http call using RestTemplate, @Async and
CompletableFuture.

@Service

class ExternalService {

private final RestTemplate restTemplate = new RestTemplate();

@Async

public CompletableFuture<User> callUserApi() {

Blocking → ~~String~~ ^{User} response = restTemplate.getForObject(url, User.class);

return CompletableFuture.completedFuture(response);

}

create a future that already finished
with its value (response)

usage:

external service. call `Users API()`

- then `Apply()`

- then `Accept()`

- exceptionally `(ex → {})`

↳ returns a fallback value

better to use `webclient`

`WebClient webClient = new WebClient.create();`

`Mono<User> response = webClient.get();`

- `uri(url)`

- `retrieve()`

- `bodyToMono(User.class);`

- more methods to handle exception in `CompletableFuture`

`future.exceptionally (ex → {
returns fallback
});`

⇒

- changes result
- handle excep
- fallback

`future.handle ((result, ex) → {
if (ex != null) {
return data transformed`

⇒

- handles success
- changes result
- handles exception
- * transform both

`future.whenComplete ((result, exp) → {});`

⇒

- don't change result
- handles exceptions
- logg

int coreCount = Runtime.getRuntime().availableProcessors()



* Deciding thread pool size.

1) CPU bound task $\Rightarrow$ no. of CPU cores.

2) IO (db, http, file) $\Rightarrow$ $\text{cores} \times (\text{waiting time} / \text{cpu time}) + 1$
 $= 4 \times (950 / 50) + 1$

$= 4 \times 20$

$= 80 \text{ threads}$

* Types of Executor thread pools

1) newFixedThreadPool :

① creates a fixed no. of worker threads. - threads don't grow beyond the defined size.

② uses an unbounded blocking queue to hold tasks

2) Single thread : newSingleThreadExecutor

① uses only one thread.

② executes tasks sequentially ensures ordering.

3) ~~cachedThreadPool~~ newCachedThreadPool

① unbounded pool $\rightarrow$ dynamically grows ^{thread.} pool

② task arrives

if thread available $\rightarrow$ reuse

else create new thread

idle thread die after 60 seconds.

- dynamically creates threads based on demand and reuses idle thread when available, uses synchronous queue so tasks are handed off directly threads instead of

bring queue.

* can create unbound no. of threads

4) `new ScheduledThreadPool`

① used for delayed and periodic tasks.

② has methods

a) `executor.scheduledelay (task, 10, seconds)`
=> task to run after 10 sec delay

b) `executor.scheduleAtFixedRate(task, 15, 10, seconds)`
=> run task repeatedly ~~after~~ every 10 seconds
initial delay 15

c) `executor.scheduleAtFixedDelay(task, 15, 10, seconds)`
=> run task repeatedly after 10 seconds after previous task completes
initial delay 10 sec

* How to webflux in project

A) Spring MVC $\Rightarrow$ tomcat, servlet, non-reactive

B) Spring webflux $\Rightarrow$ Reactive, event loop, non-blocking

If both used together (traditional enterprise)

1) spring-boot-starter-web $\rightarrow$ tomcat (servlet stack) handles controller

2) spring-boot-starter-webflux

$\rightarrow$ used as secondary library

only for webclient

if only reactive webflux

$\rightarrow$ ie springboot-starter-webflux

uses -1) netty

2) eventloop.

- reactive controllers (Mono, Flux)

Netty?

1) in traditionally spring mvc which uses one thread per request ... which might go in waiting state if response not received for IO operation, end up using more threads.

2) netty solves it by using event loop

$\rightarrow$ is asynchronous event driven reactive non blocking framework in java for high performance - high scale.

3)

I/O

- In webflux application, using netty, when a request arrives, it is handled by event loop thread that executes the controller, which may initiate any IO operation like http call using WebClient or db call using any reactive db driver. ~~any~~
- Once the IO request is made the thread is freed to handle other work. Which is not the case in MVC where Servlet thread waits and goes into waiting state till IO operation returns a response. (uses many threads)
- here in webflux we end up using less threads
- When IO response arrives, netty triggers an event and the reactive pipeline resumes, usually same event loop thread although it can be picked up by another thread and continues operation and ~~s~~ responds to client
- Achieves high concurrency with small no. of threads.

* Servlet threads in Spring MVC

default = 200

min spare thread (px created / ready) = 10

configurable

Server.tomcat.threads.max = 300

Server.tomcat.threads.min-spare = 200

* webflux $\Rightarrow$ 4-8 event loop thread. (no. of cpu cores)
reactor-netty-io.workerCount = 8

WebClient webClient = WebClient.builder()

- baseUri(uri)
- defaultHeaders(^{contentType} contentType, json)
- build();

Mono<User> response = webClient.get()

- uri("/users/1")
- retrieve()
- bodyToMono(User.class);

Error handling

• retrieve()

(1) • onStatus()

status → status.is4xx(ClientError),

response → Mono.error(new Exp());

)

• bodyToMono(User.class);

(2) onErrorResume(ex → {})

(3) doOnError(ex → log());

↳

• timeout(Duration.ofSeconds(3)) → if no response in 3 sec error

Retry

- `retry(3)` → if failed retry 3 times
- `retryWhen(Retry.fixedDelay(3, Duration.ofSeconds(3)))`
→ retry 3 time, wait 3 sec bet

Thread sends request → ^{*} goes free → ^{*} response comes → event resume
⏟
: reactive

if used in Spring MVC ... still blocking but provides

- 1) chaining
- 2) built-in retry
- 3) timeout handling
- 4) filters

Reverse proxy : ① a server that sits in front of application server

- ② intercepts incoming requests from client
- ③ load balancing : distributes incoming client requests across multiple backend servers.

upstream conf

ip1

ip2

}

proxy pass localhost

@m

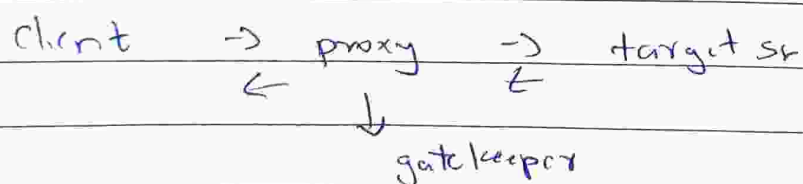
}

* round robin, least conn, ip hash (sticky), weighted.
 fewest active connections

1) ssl termination

⇒ hides the server

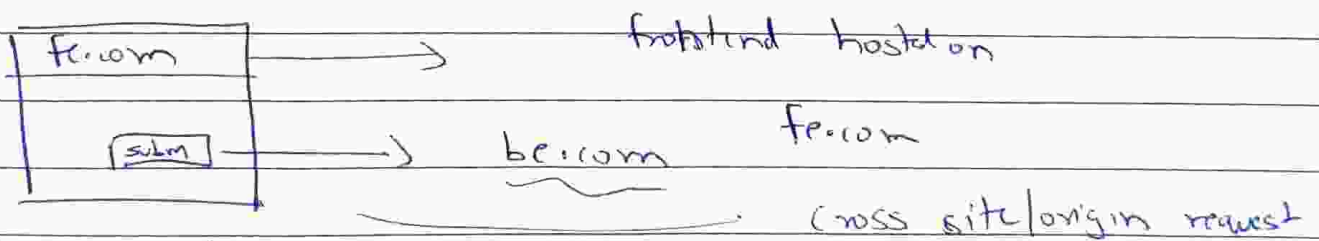
Forward Proxy : ① server that sits between client & internet



- 1) anonymity / anonyms : hides client IP
- 2) corporate filtering : within private n/w all client request will go through a proxy server
 ∴ sites can be blocked.

CORS

- 1) cross origin resource sharing security mechanism built in browser
- 2) what is origin 1) protocol (https) 2) domain 3) port
- 3) allows website from one url to access resource from/to request data from other url.
- 4) - so when a request is made from a browser a origin is set in header which is domain
 - now when it lands on a server with same origin/domain is allowed by browser
 - but if that request goes to another server (with different domain) origin mismatch is called cross origin request
- * now server will add access-control-allow-origin: some clients the lands back to client ... origin mismatch then error.



when browser sends req
it sets header origin: fe.com
so response should have same origin
Access control allow origin fe.com, *

mismatch
cross
origin
✗

Kafka:

- 1) broker: server stores data
- 2) topic: logical stream message published/consumed.
- 3) message: actual data
- 4) replication factor: how many copies

5) Partition:

- 1) topic is partitioned and distributed across brokers in round robin / key hashed.
 - 2) partition is where actual message is located.
 - 3) partition is ordered.
i.e. ~~is~~ produced & consumed in same order-process.
- ∴ so any data that has to be consumed in same order as it was produced ~~it~~ will be placed on same partition using a unique key which will be hashed.